

# GARUDA

System-on-Chip Technical Reference Manual

Prepared by: Team AeroSoC

## REVISION HISTORY:

| Version | Date       | Author       | Description                                                                                           |
|---------|------------|--------------|-------------------------------------------------------------------------------------------------------|
| V1      | -          | Team AeroSoC | Initial architecture specification                                                                    |
| V2      | 03-03-2026 | Team AeroSoC | Complete architectural redesign — updated DSU, memory subsystem, custom ISA, full autonomous pipeline |
| V3      | 19-03-2026 | Team AeroSoC | Updated Interrupt Architecture and Source of AHB-Lite                                                 |
|         |            |              |                                                                                                       |

# TABLE OF CONTENTS:

|       |                                     |
|-------|-------------------------------------|
| I.    | Introduction                        |
| II.   | System Overview                     |
| III.  | Bus Architecture                    |
| IV.   | Pin Allocation                      |
| V.    | Processor Core                      |
| VI.   | Domain-Specific Unit (DSU)          |
| VII.  | Custom Instruction Set Architecture |
| VIII. | Autonomous Flight Pipeline          |
| IX.   | Memory Architecture                 |
| X.    | DMA Controller                      |
| XI.   | Interrupt Architecture              |
| XII.  | Timers                              |
| XIII. | Peripheral Interfaces               |
| XIV.  | Communication Subsystem             |
| XV.   | Debug Interface                     |
| XVI.  | Proposals                           |

# I. INTRODUCTION:

## I.I Purpose:

This document is the architecture specification for the GARUDA SoC. It covers every hardware subsystem like processor, memory, accelerator, peripherals, and interfaces. The hardware team builds to this spec. The firmware team codes against it.

## I.II Scope:

GARUDA is a drone flight controller SoC. Not an add-on. It replaces the entire flight controller board with a single chip that handles sensing, processing, decision-making, and motor control.

The core problem: in a drone swarm, every drone needs to avoid every other drone, in real time, without a central coordinator. GARUDA uses the Artificial Potential Field (APF) algorithm to do this — each drone computes repulsion vectors from its neighbors and adjusts its trajectory accordingly. This requires fast, deterministic math on a tight power budget.

## I.III Target Application:

Quadcopter swarm drones. Up to 20 agents in close formation. Each drone runs its own instance of GARUDA. The chip handles the full autonomy stack: sensor fusion (9-state EKF), collision avoidance (APF), attitude control (PID), and path planning — all in a single 1000Hz control loop.

Hard requirement: sub-5ms end-to-end latency from sensor read to motor output. GARUDA achieves approximately 0.091ms at 200MHz.

## I.IV Design Constraints:

| PARAMETER        | VALUE                                                |
|------------------|------------------------------------------------------|
| Technology Node  | 65nm                                                 |
| Target frequency | 200 MHz (100 MHz fallback)                           |
| Die size         | 1.45 mm                                              |
| Pin count        | 40                                                   |
| Acceleration     | ISA extensions only — no separate accelerator blocks |
| OS support       | FreeRTOS                                             |
| Privilege mode   | M-mode                                               |

## **I.V Key Decisions:**

Three choices define this architecture:

First — acceleration is done through custom instructions baked into the CPU pipeline, not through a separate accelerator. We embedded a 3-MAC Domain-Specific Unit directly in the execute stage, accessible via RISC-V Custom-0 opcodes. Zero bus overhead. Deterministic latency.

Second — no caches. All memory is scratchpad (TCM) with bank-level arbitration. Caches introduce unpredictable miss penalties. For a collision avoidance system where timing jitter can cause a crash, predictable memory access is not optional.

Third — everything runs on one core. No multicore, no offloading. The full autonomy pipeline fits in ~18,200 cycles out of 200,000 available per loop iteration. One core is enough.

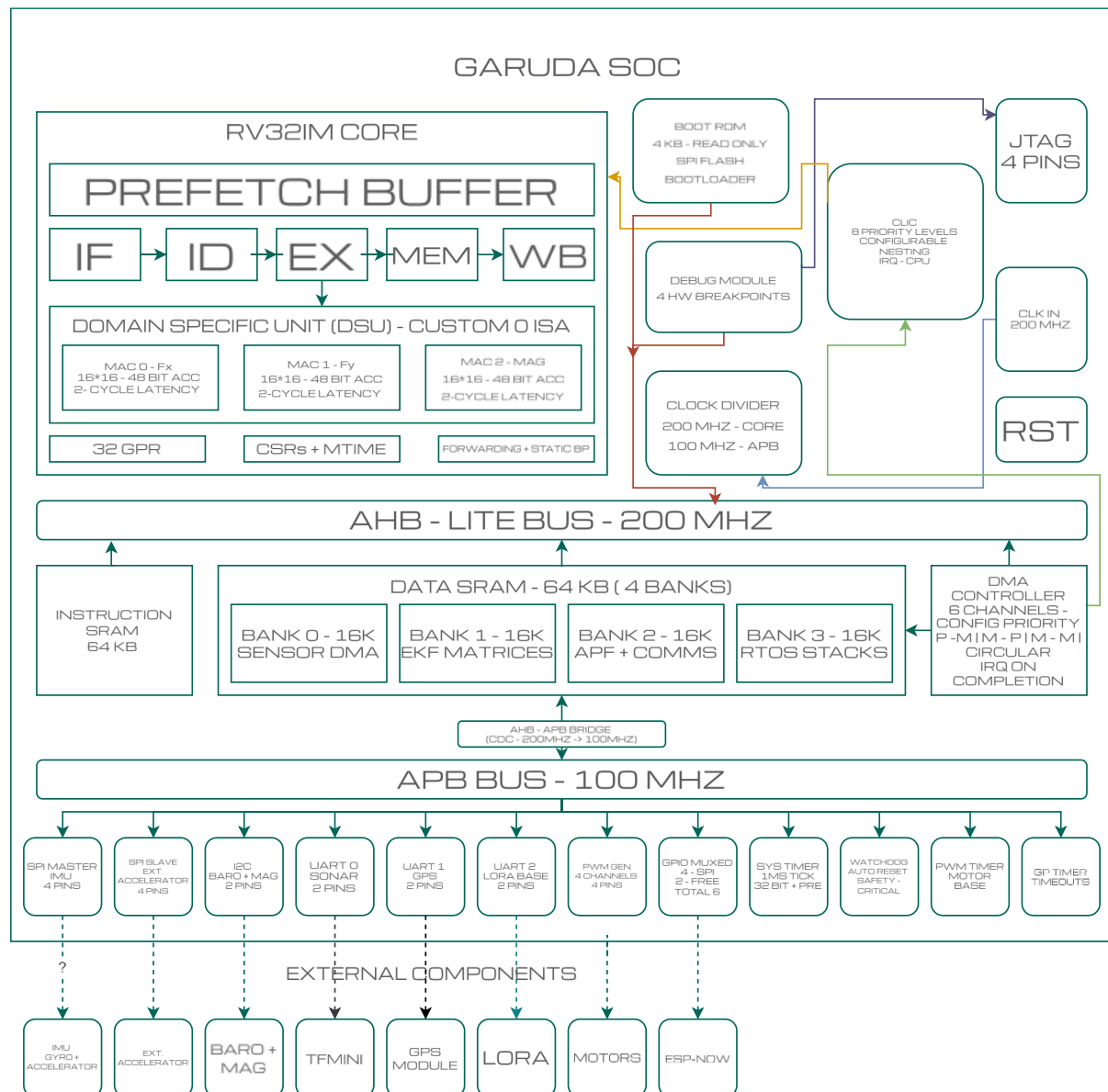
## **II. SYSTEM OVERVIEW:**

### **II.I Architecture Summary:**

GARUDA is a single-core RV32IM SoC built around a 5-stage in-order pipeline with a custom Domain-Specific Unit for collision avoidance math. It uses a two-tier bus — AHB-Lite for fast paths, APB for peripherals — with Harvard memory split into banked scratchpad. The chip runs FreeRTOS in M-mode.

Everything the drone needs to fly autonomously is on this chip. There is no external processor. The only external components are the sensors, radio modules, and motor ESCs.

## II.II Top-Level Block Diagram Components:



## III. BUS ARCHITECTURE:

### III.I Bus Topology:

Two-tier bus. AHB-Lite for high-bandwidth paths (CPU, SRAM, DMA). APB for peripherals. AHB-to-APB bridge handles protocol translation and clock domain crossing between the 200MHz core domain and 100MHz peripheral domain. The AHB-Lite bus controller is custom RTL designed to the ARM AMBA 2.0 AHB-Lite specification (IHI 0033). The AHB-to-APB bridge follows AMBA 3.0 APB (IHI 0024B). SPI, I2C, and APB peripheral IPs are provided by the VLSI Society.

### III.II AHB-Lite Bus (200 MHz):

Masters: CPU instruction port, CPU data port, DMA controller. Slaves: Instruction SRAM (64KB), Data SRAM (64KB, 4 banks), Boot ROM (4KB), APB Bridge.

AHB-Lite chosen over full AHB — single CPU core means multi-master arbitration is unnecessary. When CPU and DMA target the same SRAM bank, DMA wins, CPU stalls one cycle. Different banks — zero stall.

### III.III APB Bus (100 MHz):

All peripherals sit on APB: SPI Master, SPI Slave, I2C, UART×3, PWM, GPIO, Timers×4, CLIC, PLIC. Each peripheral gets a 4KB address window. Half-speed clock saves power on slow peripheral accesses.

Peripheral access cost from CPU: ~3-4 cycles including bridge CDC latency.

### III.IV Memory Map:

| REGION           | BASE ADDRESS | SIZE  | BUS |
|------------------|--------------|-------|-----|
| Instruction SRAM | 0x0000_0000  | 64 KB | AHB |
| Boot ROM         | 0x1000_0000  | 4 KB  | AHB |
| Data SRAM        | 0x2000_0000  | 64 KB | AHB |
| Peripherals      | 0x4000_0000  | -     | APB |

## IV. PIN ALLOCATION:

### IV.I Pin Budget:

40 pins total. 8 power, 6 infrastructure, 26 I/O.

### IV.II Pin Map:

| PIN(S)                  | COUNT | FUNCTION              | DIRECTION |
|-------------------------|-------|-----------------------|-----------|
| VDD                     | 4     | Power supply          | -         |
| VSS                     | 4     | Ground                | -         |
| CLK                     | 1     | Clock input (200 MHz) | Input     |
| RST                     | 1     | Active-low reset      | Input     |
| TCK, TMS, TDI, TDO      | 4     | JTAG debug            | Mixed     |
| SPI_SCK, MOSI, MISO, CS | 4     | SPI Master — IMU      | Output/IO |

|                          |    |                                      |               |
|--------------------------|----|--------------------------------------|---------------|
| SPIS_SCK, MOSI, MISO, CS | 4  | SPI Slave — Ext Accelerator          | Input/IO      |
| SDA, SCL                 | 2  | I2C — Barometer, Magnetometer        | Bidirectional |
| UART0_TX, UART0_RX       | 2  | SONAR                                | Output/Input  |
| UART1_TX, UART1_RX       | 2  | GPS                                  | Output/Input  |
| UART2_TX, UART2_RX       | 2  | LoRa (Base-to-Drone)                 | Output/Input  |
| PWM0–PWM3                | 4  | Motor ESCs (Quadcopter)              | Output        |
| GPIO0–GPIO3              | 4  | Muxed — ESP-NOW SPI (Drone-to-Drone) | Configurable  |
| GPIO4–GPIO5              | 2  | Muxed — Free/General purpose         | Configurable  |
| Total                    | 40 |                                      |               |

#### IV.III Pin Muxing:

GPIO0–GPIO5 support alternate functions configurable via the GPIO control register. GPIO0–3 can be muxed as a second SPI port for the ESP-NOW radio module. GPIO4–5 are available for future use — LEDs, status signals, or additional peripheral interfaces.

## V.PROCESSOR CORE:

#### V.I Overview:

Custom-built RV32IM core. 5-stage in-order pipeline. Not based on Ibex or any existing core — designed from scratch to integrate the DSU directly into the execute stage.

#### V.II Pipeline:

| STAGE | FUNCTION                                            |
|-------|-----------------------------------------------------|
| IF    | Instruction fetch from ISRAM via prefetch buffer    |
| ID    | Decode + register file read + hazard detection      |
| EX    | ALU / Multiply / DSU operations / Branch resolution |
| MEM   | Data memory load/store                              |
| WB    | Write-back to register file                         |



### **V.III ISA Support:**

Base: RV32I (integer). Extension: M (hardware multiply, software-only divide). Custom: Custom-0 opcode space for DSU instructions. Privilege: M-mode only.

### **V.IV Hazard Handling:**

Full data forwarding with two paths: EX→EX and MEM→EX. Eliminates most data hazard stalls. Load-use hazard inserts a one-cycle bubble — unavoidable since the loaded value isn't available until end of MEM stage.

### **V.V Branch Handling**

Static branch prediction — backward taken, forward not taken. Branches resolve in EX stage. Misprediction penalty: 2 cycles (flush IF and ID). For the APF inner loop (backward branch, iterating over neighbors), prediction is correct every iteration except the last.

### **V.VI Multiplier**

Single-cycle 32×32 hardware multiplier in the EX stage. Supports MUL, MULH, MULHSU, MULHU. Single-cycle was chosen because APF and EKF are multiply-heavy workloads — a multi-cycle multiplier would bottleneck the critical path.

### **V.VII Divider**

Software only. DIV and REM instructions trap to a software handler using Newton-Raphson reciprocal approximation (~5-10 multiply instructions). Division is rare in APF math. Hardware divider area is better spent elsewhere.

### **V.VIII Register File**

32 general-purpose registers (x0–x31), standard RV32. Two read ports, one write port. x0 hardwired to zero.

### **V.IX Prefetch Buffer**

2-4 instruction lookahead buffer between ISRAM and IF stage. Smooths fetch latency and keeps the pipeline fed during sequential execution. Not a cache — no tag matching, no replacement policy. Just a FIFO.

### **V.X CSRs**

Standard M-mode CSRs: mstatus, mie, mtvec, mepc, mcause, mtval, mip, mcycle, minstret. Plus mtime/mtimecmp for FreeRTOS tick. DSU accumulators are accessed through custom instructions, not CSRs.

## VI. DOMAIN-SPECIFIC UNIT (DSU):

### VI.I Overview:

3 MAC units embedded directly in the EX stage. Not a separate accelerator — it's an ISA extension using the RISC-V Custom-0 opcode space. Zero bus overhead. The CPU executes DSU instructions the same way it executes ADD or MUL.

### VI.II MAC Specifications:

Each MAC performs a  $16 \times 16$  signed multiply and accumulates the 32-bit product into a 48-bit accumulator. Two-cycle latency — cycle 1 does the multiply, cycle 2 does the accumulate. Three MACs, one per component: MAC 0 for  $F_x$  (repulsion  $X$ ), MAC 1 for  $F_y$  (repulsion  $Y$ ), MAC 2 for Magnitude. All three can fire in parallel or be addressed individually.

### VI.III Pipeline Integration:

DSU instructions take 2 cycles in the EX stage. The pipeline stalls for the extra cycle. Interlock (allowing non-dependent instructions to proceed) is a proposal for future implementation. DSU results are read back to GPRs through dedicated MACREAD instructions.

### VI.IV Accumulator Access:

Accumulators are 48-bit but GPRs are 32-bit. Read is split: MACREAD\_LO returns the lower 32 bits, MACREAD\_HI returns the upper 16 bits. Accumulators are cleared with MACCLEAR before each new computation (e.g., before processing a new set of neighbors).

### VI.V Custom Instruction Set:

10 instructions, all encoded in Custom-0 opcode space (0001011):

- MAC\_SEL  $rs1, rs2, acc$  — multiply  $rs1 \times rs2$  and accumulate into selected accumulator.
- MACSUB  $rs1, rs2, acc$  — multiply  $rs1 \times rs2$  and subtract from selected accumulator. Used for attractive field computation.
- MACCLEAR  $acc$  — clear selected accumulator to zero. Pass 0x3 to clear all three.
- MACREAD\_LO  $rd, acc$  — read lower 32 bits of accumulator into  $rd$ .
- MACREAD\_HI  $rd, acc$  — read upper 16 bits of accumulator into  $rd$ .
- MACLOAD  $acc, rs1$  — load a 32-bit value directly into an accumulator.
- MACDOT  $rs1, rs2, acc$  — dot product mode. Two multiply-adds in one instruction for matrix operations. Primary use: EKF matrix multiply.
- MACSHIFT  $acc, imm$  — arithmetic right shift accumulator by  $imm$  bits. Used for fixed-point scaling after accumulation.
- MACSAT  $acc$  — saturate accumulator value to 32-bit signed range. Clamps overflow.
- MACABS  $rs1, rs2, acc$  — multiply-accumulate using absolute values of operands. Used for magnitude calculations.

## VII. AUTONOMOUS FLIGHT PIPELINE:

### VII.I Overview:

The entire autonomous stack runs on-chip in a single 1000Hz control loop. No external processor. Five stages execute sequentially every 1ms.

### VII.II Stage 1 – Sensor Fusion (9-State EKF):

Fuses IMU, barometer, magnetometer, GPS, and SONAR into a clean state estimate. Tracks 9 variables: 3 position, 3 velocity, 3 gyro biases. Core math is matrix multiply and inversion on 9×9 matrices — accelerated by DSU MACDOT instructions. Approximately 12,000 cycles per iteration.

### VII.III Stage 2 – Path Planning:

Receives waypoints from the base station via LoRa. Interpolates a smooth trajectory between waypoints and outputs a target position and velocity vector. Approximately 3,000 cycles.

### VII.IV Stage 3 – Collision Avoidance (APF):

Receives neighbor drone positions via ESP-NOW. For each neighbor, computes a repulsion vector using the Artificial Potential Field algorithm. The destination generates an attraction vector. All vectors are summed into a final adjusted velocity command. This is the DSU's primary workload — MAC\_SEL computes  $dx \times scale$  and  $dy \times scale$  for each neighbor, MACSUB handles attractive forces. Approximately 300 cycles for 10 neighbors.

### VII.V Stage 4 – Flight Control (PID):

Four PID loops: roll, pitch, yaw, altitude. Takes the difference between desired state (from APF output) and actual state (from EKF output) and computes motor thrust adjustments. Multiply-add heavy — DSU helps here too. Approximately 300 cycles.

### VII.VI Stage 5 – Motor Output:

Motor mixer converts 4 PID outputs into individual motor speeds. PWM hardware drives the ESCs. Approximately 100 cycles.

### VII.VII Cycle Budget:

Total: approximately 18,200 cycles per loop. Budget at 200MHz/1000Hz: 200,000 cycles. Headroom: 91%. Achieved latency: 0.091ms — well under the 5ms requirement.

## VIII. MEMORY ARCHITECTURE:

### VIII.I Overview:

Harvard architecture — separate instruction and data buses. All scratchpad (TCM), no caches. Deterministic access times for safety-critical real-time operation.

### VIII.II Instruction SRAM:

64KB, single-port, on the AHB instruction bus. Holds the full firmware — FreeRTOS kernel, EKF, APF, PID, drivers, comms stack. Loaded from external SPI flash at boot by the Boot ROM bootloader.

### VIII.III Data SRAM:

64KB, 4 banks of 16KB each, on the AHB data bus. Bank 0 is for sensor DMA buffers. Bank 1 holds EKF matrices and state vectors. Bank 2 holds APF neighbor data and comms buffers. Bank 3 holds FreeRTOS task stacks and heap. Banking allows DMA and CPU to access different banks simultaneously with zero stall.

### VIII.IV Boot ROM:

4KB, read-only, hardwired. Contains the bootloader that reads firmware from external SPI flash and loads it into instruction SRAM. Reset vector points here.

### VIII.V Prefetch Buffer:

2-4 instruction FIFO between ISRAM and the IF stage. Not a cache. Keeps the pipeline fed during sequential execution.

## IX. DMA CONTROLLER:

### IX.I Overview:

6-channel DMA with configurable priority arbitration. Offloads all sensor data transfers from the CPU. Fires an interrupt on completion.

### IX.II Channels:

CH0: SPI Master (IMU). CH1: I2C (Barometer, Magnetometer). CH2: UART0 (SONAR). CH3: UART1 (GPS). CH4: UART2 (LoRa). CH5: GPIO/SPI (ESP-NOW).

### IX.III Transfer Modes:

Peripheral-to-Memory for sensor reads. Memory-to-Peripheral for radio TX and PWM updates. Memory-to-Memory for buffer copying and double-buffering EKF data. Circular mode for continuous sensor streaming — DMA auto-restarts when a transfer completes, no CPU intervention needed.

## IX.IV Arbitration:

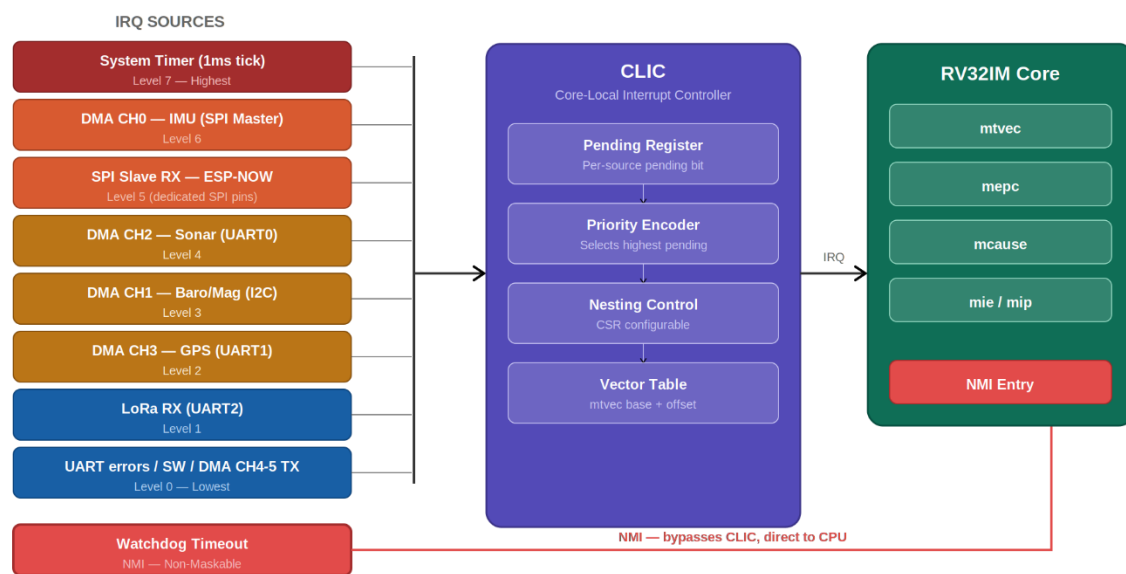
Software-configurable priority per channel. Default: IMU highest, GPS lowest. Priority can be changed at runtime for different flight phases.

# X. INTERRUPT ARCHITECTURE:

## X.I Controllers:

CLIC as the primary interrupt controller. 8 priority levels. Configurable nesting — enable or disable via CSR.

## X.II Priority Assignment:



Level 7 (highest): System timer tick (1ms control loop).

Level 6: DMA complete — IMU.

Level 5: ESP-NOW RX (neighbor data for APF).

Level 4: DMA complete — SONAR.

Level 3: DMA complete — Barometer/Magnetometer.

Level 2: DMA complete — GPS.

Level 1: LoRa RX (base station commands).

Level 0 (lowest): UART errors, software interrupts.

## XI. TIMERS:

System timer: 32-bit with prescaler, generates the 1ms control loop tick. Watchdog timer: resets the SoC if firmware hangs — mandatory for a flying drone. PWM timer: dedicated timebase for motor PWM signal generation. General purpose timer: available for sensor timeouts, comms timeouts, and performance profiling. All timers are 32-bit with configurable prescalers. RISC-V mtime/mtimecmp pair is implemented separately as required by FreeRTOS.

## XII. PERIPHERAL INTERFACES:

SPI Master: connects to IMU (Gyro + Accelerometer) on a dedicated bus. No sharing — IMU is the most timing-critical sensor. SPI Slave: exposes an interface for an external accelerator (FPGA or ASIC). Gives users a growth path beyond the on-chip DSU. I2C: shared bus for Barometer and Magnetometer. Both are slow-update sensors so bus sharing is fine. UART0: full duplex, connected to TFMini LiDAR/SONAR for obstacle detection. UART1: full duplex, connected to GPS module for position data. UART2: full duplex, connected to LoRa radio for base station communication. PWM Generator: 4 channels, one per motor ESC. Quadcopter configuration. GPIO: 6 pins with muxing support. GPIO0-3 mux as SPI for the ESP-NOW radio module (drone-to-drone comms). GPIO4-5 free for general use.

## XIII. COMMUNICATION SUBSYSTEM:

Drone-to-Drone: ESP-NOW protocol on an ESP32 module. Broadcast mode — no pairing needed. Each drone broadcasts its position and velocity to all neighbors. Latency approximately 1-5ms per packet. Connected via SPI through muxed GPIO pins.

Base-to-Drone: LoRa (SX1276) for long-range command and waypoint delivery from the ground station. Range 1km+. Connected via UART2. Low bandwidth, infrequent updates — waypoints and mission commands only.

Separate radios for each link. If one fails, the other still works.

## XIV. DEBUG INTERFACE:

RISC-V Debug Module v0.13 over JTAG (4 pins: TCK, TMS, TDI, TDO). 4 hardware breakpoints. Supports halt, single-step, resume, and full register + memory access while halted. Compatible with OpenOCD and GDB. Used for firmware development and post-silicon bring-up.

## XV. PROPOSALS:

These are features designed but deferred. Implement if schedule and die area permit.

**384KB TCM:** Expand memory from 128KB to 384KB for larger firmware, more swarm neighbors, or higher-fidelity EKF.

**MAC\_ALL instruction:** Fire all 3 MACs in parallel using pre-loaded shadow registers. Requires 6 additional shadow registers in the DSU. Performance benefit is marginal with 2-cycle MACs but becomes significant if the MAC is later optimized to single-cycle.

**Pipeline interlock for DSU:** Allow non-DSU instructions to proceed while a 2-cycle MAC operation completes. Replaces the current stall-based approach. Cost is approximately 20-30 gates. Benefit is modest for back-to-back DSU instructions but helps mixed code sequences.